Pandas Series & DataFrames

Complete Masterclass Notes

Data Science Study Guide

October 7, 2026

Contents

1 Module 1: Introduction to the Environment **1**
1.1 Importing Pandas 2

2 Module 2: Core Data Structures **2**
2.1 Creating a DataFrame from Scratch 2
2.2 Loading Built-in Data (Scikit-Learn) 2

3 Module 3: Data Exploration & Inspection **3**
3.1 Viewing Data & Understanding Structure 3
3.2 Deep Dive into Data Types and Nulls 3
3.3 Summary Statistics 3

4 Module 4: Data Selection and Indexing **3**
4.1 Selecting Columns 3
4.2 The Ultimate Showdown: `.loc` vs `.iloc` 4

5 Module 5: Conditional Slicing (Filtering) **4**
5.1 Basic Boolean Masking and Bitwise Operators 4
5.2 Advanced: Lambda Functions in `.loc` 5

1 Module 1: Introduction to the Environment

Before diving into Pandas, the lectures introduced **Google Colab**, a cloud-based Jupyter Notebook environment.

- **Zero Configuration:** Pre-installed with major Machine Learning packages.
- **Cell Types:**
  - **Code Cells:** For executing Python code (Run using `Shift + Enter`).
  - **Text/Markdown Cells:** For writing documentation. Supports Markdown and LaTeX for mathematical equations, such as linear regressions:

$$Y = w_0 + w_1 X_1 + \cdots + w_n X_n$$

1.1 Importing Pandas

By convention, Pandas is almost always imported with the alias `pd`, and NumPy as `np`.

```
import pandas as pd
import numpy as np

# Check your pandas version
print(pd.__version__)
```

2 Module 2: Core Data Structures

Pandas is a **column-oriented** data analysis API. It fundamentally relies on two structural concepts:

Core Concept

1. **Series (1D):** A single column of data with an associated array of data labels, called an *index*.
2. **DataFrame (2D):** A tabular structure consisting of rows and columns. Mathematically, think of it as an $m \times n$ matrix where m is rows and n is columns. A DataFrame is a collection of Series sharing the same index.

2.1 Creating a DataFrame from Scratch

You can create a DataFrame by passing a Python dictionary where keys act as column names and values are `pd.Series` or standard lists.

```
# 1. Create Series objects
city_series = pd.Series(['Mumbai', 'Bangalore', 'Chennai'])
pop_series = pd.Series([17000000, 13000000, 6000000])

# 2. Combine into a DataFrame
city_info_df = pd.DataFrame({
    'City': city_series,
    'Population': pop_series
})

print(city_info_df)
```

Pro-Tip

If you add a 4th city to the `city_series` but don't provide a 4th population in `pop_series`, Pandas won't crash! It gracefully aligns the indices and fills the missing population value with `NaN` (Not a Number).

2.2 Loading Built-in Data (Scikit-Learn)

Instead of manually typing data, it's common to load pre-existing datasets. The lectures use the `diabetes` dataset from `sklearn`.

```
from sklearn.datasets import load_diabetes

# The as_frame=True argument ensures the data is a Pandas DataFrame
diabetes_data = load_diabetes(as_frame=True)
df = diabetes_data.data
```

3 Module 3: Data Exploration & Inspection

Once a DataFrame is loaded, your first step is exploration to understand the data's structure and statistics.

3.1 Viewing Data & Understanding Structure

- `df.head(n)`: Returns the first n rows (default $n = 5$).
- `df.tail(n)`: Returns the last n rows.
- `df.shape`: Returns a tuple (m, n) representing rows and columns.
- `df.columns`: Returns a list-like object of all column names.

3.2 Deep Dive into Data Types and Nulls

The `.info()` method provides critical operational metadata:

- Total number of rows and index range.
- Count of non-null values per column (vital for spotting missing data).
- The data type of each column (e.g., `float64`, `int64`, `object`).
- Memory usage footprint.

3.3 Summary Statistics

To catch outliers and understand the distribution of numerical data, use `.describe()`. It calculates statistical properties like the mean (μ), standard deviation (σ), minimum, maximum, and quantiles.

```
# View default statistics for numerical columns
df.describe()

# Request custom percentiles
df.describe(percentiles=[0.20, 0.50, 0.80])

# Transpose (.T) the result for easier row-by-row reading
df.describe().T
```

4 Module 4: Data Selection and Indexing

Accessing specific rows, columns, or singular cells is a core competency in Pandas.

4.1 Selecting Columns

Treat the DataFrame like a dictionary where column names are the keys.

```
# Select a single column (Returns a Series)
age_series = df['age']

# Select multiple columns (Returns a subset DataFrame)
# Note the double brackets [[ ]] - you are passing a list of keys
subset_df = df[['age', 'sex', 'bmi']]
```

4.2 The Ultimate Showdown: `.loc` vs `.iloc`

Students frequently confuse these two indexers. Here is the definitive guide:

Feature	`df.loc[]` (Location)	`df.iloc[]` (Integer Location)
Search Method	By **Label / Name** (Index names, Column names)	By **Integer Position** (0, 1, 2...)
Row Example	`df.loc['Row_A']`	`df.iloc[0]`
Column Example	`df.loc[:, 'age']`	`df.iloc[:, 0]`
End Bound	**Inclusive** (e.g., `'a':'c'` includes 'c')	**Exclusive** (e.g., `0:3` stops at 2)

```python
# --- ILOC (Strictly Numbers!) ---
# Get the first row (index position 0)
first_row = df.iloc[0]

# Get the first 4 rows and first 2 columns using standard python slicing
slice_df = df.iloc[0:4, 0:2]

# --- LOC (Strictly Labels!) ---
# Assuming your DataFrame has string labels for rows like 'a', 'b', 'c'
# Get rows from 'a' to 'd' and specific columns
subset = df.loc['a':'d', ['age', 'bmi']]
```

Pro-Tip

If your DataFrame uses a default numeric index (0, 1, 2...), `loc[0]` and `iloc[0]` might appear identical. However, `loc[0]` searches for the literal string/name "0", while `iloc[0]` searches for the positional offset 0.

5 Module 5: Conditional Slicing (Filtering)

Data is rarely useful without filtering. You will frequently want to extract rows based on specific mathematical or logical conditions.

5.1 Basic Boolean Masking and Bitwise Operators

If you have multiple conditions, you **must** wrap each condition in parentheses `()` and use bitwise operators. Standard Python `and`/`or` keywords will raise an error.

- Logical AND: `&` (Intersection: $A \wedge B$)
- Logical OR: `|` (Union: $A \vee B$)
- Logical NOT: `~` (Negation: $\neg A$)

```python
# Find all records where Age is less than a specific value
young_patients = df[df['age'] < -0.05]

# Example: Age > 0.05 AND BMI < 0.02
condition1 = df['age'] > 0.05
condition2 = df['bmi'] < 0.02

# Apply the bitwise AND filter
filtered_df = df[(condition1) & (condition2)]

# Example using NOT (~) and OR (|)
complex_filter = df[~(condition1) | (condition2)]
```

5.2 Advanced: Lambda Functions in `.loc`

You can pass a nameless, inline function (a `lambda` function) into `.loc`. The lambda function receives the DataFrame as an input and evaluates to a boolean condition for filtering.

```python
# Select all rows where column 'a' is greater than 0.5
# lambda d: d['a'] > 0.5 dynamically generates a boolean mask
result = df.loc[lambda d: d['a'] > 0.5]

# Complex lambda with AND
result_complex = df.loc[lambda d: (d['a'] > 0.5) & (d['b'] < 0.2)]
```

Core Concept

Why use Lambda here?
If you are processing data through a chain of methods (e.g., `df.dropna().assign(new_col=...).loc[...]`), a lambda function inside `.loc` allows you to refer to the *current, intermediate* state of the DataFrame without needing to save it to a temporary variable first!